
Integration Manual

for MPC560XB PORT Driver

Document Number: IM13PORTASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	9
2.2	Overview.....	10
2.3	About this Manual.....	10
2.4	Acronyms and Definitions.....	10
2.5	Reference List.....	11
Chapter 3		
Building the Driver		
3.1	Build Options.....	13
3.1.1	CW Compiler/Linker/Assembler Options.....	13
3.1.2	DIAB Compiler/Linker/Assembler Options.....	15
3.1.3	GHS Compiler/Linker/Assembler Options.....	17
3.1.4	HT Compiler/Linker/Assembler Options.....	18
3.2	Files required for Compilation.....	19
3.3	Setting up the Plug-ins.....	21
Chapter 4		
Function calls to module		
4.1	Function Calls during Start-up.....	23
4.2	Function Calls during Shutdown.....	23
4.3	Function Calls during Wake-up.....	23
Chapter 5		
Module requirements		
5.1	Exclusive areas to be defined in BSW scheduler.....	25
5.2	Peripheral Hardware Requirements.....	25
5.3	ISR to configure within OS – dependencies.....	25

Section number	Title	Page
5.4	ISR Macro.....	25
5.5	Other AUTOSAR modules - dependencies.....	25
Chapter 6 Main API Requirements		
6.1	Main functions calls within BSW scheduler.....	27
6.2	API Requirements.....	27
6.3	Calls to Notification Functions, Callbacks, Callouts.....	27
Chapter 7 Memory Allocation		
7.1	Sections to be defined in MemMap.h.....	29
7.2	Linker command file.....	30
Chapter 8 Configuration parameters considerations		
8.1	Configuration Parameters.....	31
Chapter 9 Integration Steps		
Chapter 10 ISR Reference		
10.1	Software specification.....	35
10.1.1	Define Reference.....	35
10.1.2	Macros Reference.....	35
10.1.2.1	Macro SIUL_GPDI.....	35
10.1.2.2	Macro SIUL_GPDO.....	35
10.1.2.3	Macro SIUL_IFMC.....	35
10.1.2.4	Macro SIUL_MPGPDO.....	36
10.1.2.5	Macro SIUL_PCR.....	36
10.1.2.6	Macro SIUL_PGPDI.....	36
10.1.2.7	Macro SIUL_PGPDO.....	36
10.1.2.8	Macro SIUL_PSMI.....	36
10.1.3	Enum Reference.....	36

Section number	Title	Page
10.1.4	Function Reference.....	36
10.1.5	Structs Reference.....	36
10.1.6	Types Reference.....	36
10.1.7	Variables Reference.....	37



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	08-Feb-2012	Alfredo Di Martino	First draft for BETA 0.9.0
1.1	13-May-2013	Tilak Basavaraju	Updated for RTM 1.0.0
1.2	02-June-2014	François Penalosa	Updated for RTM 1.0.1



Chapter 2

Introduction

This integration manual describes the integration requirements for Port Driver for MPC560XB microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as MPC5607B .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
ADC	Analog to Digital Converter
API	Application Programming Interface
ASM	Assembler
AUTOSAR	Automotive Open System Architecture
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SIUL	Systems Integration Unit Lite
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003Port Driver Software Specification Document.	4.0 Rev0003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013

Table continues on the next page...

Table 2-3. Reference List (continued)

#	Title	Version
6	Bolero 1.5M cut 2.2 Errata Sheet Reference Manual	MPC5607B_1M03Y - Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet Reference Manual	MPC5604B_2M27V - Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet Reference Manual	MPC5602D_1M18Y - Rev. , 12 DEC 2013

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar Port driver for Freescale SemiconductorMPC560XB . It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The Port driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note

The TS_T2D13M10I1R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-2. Assembler Options (continued)

Option	Description
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be

Table 3-10. Compiler Options

Option	Description
	added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like --noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script ie sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Port driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_RELEASE_MAJOR_VERSION and AR_RELEASE_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

Port Files

- ..\Port _ TS_T2D13M10I1R0 \src\Port.c
- ..\Port _ TS_T2D13M10I1R0 \src\Port_Siul_LLD.c
- ..\Port _ TS_T2D13M10I1R0 \include\Port.h
- ..\Port _ TS_T2D13M10I1R0 \include\Reg_eSys_SIUL.h
- ..\Port _ TS_T2D13M10I1R0 \include\Port_Siul_LLD.h
- ..\Port _ TS_T2D13M10I1R0 \include\Port_Siul_LLD_CfgEx.h
- ..\Port _ TS_T2D13M10I1R0 \include\Port_LLD.h

Port Generated Files

- Port_Cfg.c (For PC Variant) - This file should be generated by the user using a configuration tool for compilation.
- Port_PBcfg.c (For PB Variant) - This file should be generated by the user using a configuration tool for compilation.
- Port_Cfg.h - This file should be generated by the user using a configuration tool for compilation.

Files from Base folder

- ..\Base_ TS_T2D13M10I1R0 \include\Can_GeneralTypes.h
- ..\Base_ TS_T2D13M10I1R0 \include\Cer.h
- ..\Base_ TS_T2D13M10I1R0 \include\Compiler.h
- ..\Base_ TS_T2D13M10I1R0 \include\Compiler_Cfg.h
- ..\Base_ TS_T2D13M10I1R0 \include\ComStack_Cfg.h
- ..\Base_ TS_T2D13M10I1R0 \include\ComStack_Types.h
- ..\Base_ TS_T2D13M10I1R0 \include\Lin_GeneralTypes.h
- ..\Base_ TS_T2D13M10I1R0 \include\Mcal.h
- ..\Base_ TS_T2D13M10I1R0 \include\MemMap.h
- ..\Base_ TS_T2D13M10I1R0 \include\Platform_Types.h
- ..\Base_ TS_T2D13M10I1R0 \include\Reg_eSys.h
- ..\Base_ TS_T2D13M10I1R0 \include\Reg_Macros.h
- ..\Base_ TS_T2D13M10I1R0 \include\Soc_Ips.h
- ..\Base_ TS_T2D13M10I1R0 \include\Std_Types.h

Files from Det folder:

- ..\Det_TS_T2D13M10I1R0\include\Det.h

Note:

<plugin_name>: TS_T<2>D<13>M<SW_Version_Major>I<SW_Version_Minor>R0

(i.e. Target_Id = 2 identifies PowerPC architecture and Derivative_Id = 11 identifies the MPC560XB)

3.3 Setting up the Plug-ins

The Port driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

Location of various files inside the FR module folder:

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:
 - Port_TS_T2D13M10I1R0\config\Port.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - Port_TS_T2D13M10I1R0\autosar\Port_MPC5607B_lqfp100.epd
 - Port_TS_T2D13M10I1R0\autosar\Port_MPC5607B_lqfp144.epd
- Code Generation Templates for Pre-Compile time configuration parameters:
 - ..\Port_TS_T2D13M10I1R0\generate_PC\include\Port_Cfg.h
 - ..\Port_TS_T2D13M10I1R0\generate_PC\src\Port_Cfg.c
- Code Generation Templates for Post-Build time configuration parameters:
 - ..\Port_TS_T2D13M10I1R0\generate_PC\src\Port_PBCfg.c

Steps to generate the configuration:

1. Copy the module folders Port_TS_T2D13M10I1R0 , Base_TS_T2D13M10I1R0 , Resource_TS_T2D13M10I1R0 , EcuM_TS_T2D13M10I1R0 , Det_TS_T2D13M10I1R0 , Rte_TS_T2D13M10I1R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.



Chapter 4

Function calls to module

4.1 Function Calls during Start-up

None.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

5.1 Exclusive areas to be defined in BSW scheduler

None.

5.2 Peripheral Hardware Requirements

The PORT driver uses MPC560XB's peripheral.

5.3 ISR to configure within OS – dependencies

None.

5.4 ISR Macro

None.

5.5 Other AUTOSAR modules - dependencies

- **Det** This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation/deactivation of Development error detection is configurable using 'CanDevErrorDetect' configuration parameter.

- **EcuM:** This module is necessary for a reference to the Wakeup source for this controller as defined in the ECU State Manager.
- **Resource:** This module is required to select processor derivative. Current Port driver has support for the following derivatives, everyone having attached a Resource file:
mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64
mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100
mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100
mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144
mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64
mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208
mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100
mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176
mpc5607bxmg_mapbga208 .



Chapter 6

Main API Requirements

6.1 Main functions calls within BSW scheduler

None.

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

None.

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifndef PORT_START_CONFIG_DATA_UNSPECIFIED
    #undef PORT_START_CONFIG_DATA_UNSPECIFIED
    #undef MEMMAP_ERROR
    /*Memory Section for Post Build Data to be defined here.
    Example given in the next line*/
    #pragma ghs section const=".pbport_cfg"
    #endif
    #ifndef PORT_STOP_CONFIG_DATA_UNSPECIFIED
    #undef PORT_STOP_CONFIG_DATA_UNSPECIFIED
    #undef MEMMAP_ERROR
    /*End of section to be mentioned here. Example given in the
    next line.*/
    #pragma ghs section
    #endif
```

For Code:

```
#ifndef PORT_START_SEC_CODE
    #undef PORT_START_SEC_CODE
    #undef MEMMAP_ERROR
    /*Memory Section for Code to be defined here.*/
    #endif
    #ifndef PORT_STOP_SEC_CODE
    #undef PORT_STOP_SEC_CODE
    #undef MEMMAP_ERROR
    /*End of section to be mentioned here*/
    #endif
```

For Variables:

```
#ifndef PORT_START_SEC_VAR_UNSPECIFIED
    #undef PORT_START_SEC_VAR_UNSPECIFIED
    #undef MEMMAP_ERROR
    /*Memory Section for Variables to be defined here.*/
    #endif
    #ifndef PORT_STOP_SEC_VAR_UNSPECIFIED
```

Linker command file

```
#undef PORT_STOP_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Constant data:

```
#ifdef PORT_START_SEC_CONST_UNSPECIFIED
#undef PORT_START_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*Memory Section for Constants to be defined here.*/
#endif
#ifdef PORT_STOP_SEC_CONST_UNSPECIFIED
#undef PORT_STOP_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration parameters considerations

Configuration parameter class for Autosar Port driver fall into the following variants as defined below:

8.1 Configuration Parameters

Specifies whether the configuration parameter shall be of configuration class Post Build

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
PortGeneral	PortDevErrorDetect	Pre Compile Parameter for all configuration variants	Pre Compile
	PortVersionInfoApi	Pre Compile Parameter for all configuration variants	Pre Compile
	PortSetPinDirectionApi	VariantPC or VariantPB	Pre Compile
	PortSetPinModeApi	VariantPC or VariantPB	Pre Compile
	PortReversePortBits	VariantPC or VariantPB	Pre Compile
DefaultPortPin	PortPinInitialMode	VariantPC or VariantPB	Pre Compile
	PortPinWpe	VariantPC or VariantPB	Pre Compile
	PortPinReadback	VariantPC or VariantPB	Pre Compile
	PortPinSafeMode	VariantPC or VariantPB	Pre Compile
	PortPinWps	VariantPC or VariantPB	Pre Compile
	PortPinDirection	VariantPC or VariantPB	Pre Compile
	PortPinLevelValue	VariantPC or VariantPB	Pre Compile
	PortPinLevelValue	VariantPC or VariantPB	Pre Compile
PortContainer	PortNumberOfPortPins	VariantPC or VariantPB	Pre Compile
	PortPinWpe	VariantPC or VariantPB	Pre Compile
	PortPinWps	VariantPC or VariantPB	Pre Compile
	PortPinOde	VariantPC or VariantPB	Pre Compile
	PortPinSafeMode	VariantPC or VariantPB	Pre Compile
	PortPinDirectionChangeable	VariantPC or VariantPB	Pre Compile
	PortPinReadback	VariantPC or VariantPB	Pre Compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	PortPinModeChangeable	VariantPC or VariantPB	Pre Compile
	PortPinId	VariantPC or VariantPB	Pre Compile
	PortPinPcr	VariantPC or VariantPB	Pre Compile
	PortPinDirection	VariantPC or VariantPB	Pre Compile
	PortPinInitialMode	VariantPC or VariantPB	Pre Compile
	PortPinMode	VariantPC or VariantPB	Pre Compile
	PortPinLevelValue	VariantPC or VariantPB	Pre Compile
	PortPinSlewRate	VariantPC or VariantPB	Pre Compile
CommonPublishedInformation	VendorId	VariantPC or VariantPB	Pre Compile
	ModuleId	VariantPC or VariantPB	Pre Compile
	ArReleaseMajorVersion	VariantPC or VariantPB	Pre Compile
	ArReleaseMinorVersion	VariantPC or VariantPB	Pre Compile
	ArReleaseRevisionVersion	VariantPC or VariantPB	Pre Compile
	SwMajorVersion	VariantPC or VariantPB	Pre Compile
	SwMinorVersion	VariantPC or VariantPB	Pre Compile
	SwPatchVersion	VariantPC or VariantPB	Pre Compile
	VendorApilInfix	VariantPC or VariantPB	Pre Compile

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating Port Driver :

- Generate the required PORT configurations.
- Allocate proper memory sections in MemMap.h and linker command file.
- Make sure all include files for compilation are involved as described in section [Files required for Compilation](#).
- Compile and build the PORT module with all the dependent modules. For more details refer to section [Other AUTOSAR modules - dependencies](#).



Chapter 10

ISR Reference

ISR functions exported by the Port driver.

10.1 Software specification

The following sections contains driver software specifications.

10.1.1 Define Reference

Constants supported by the Port driver.

10.1.2 Macros Reference

Macros supported by the Port driver.

10.1.2.1 Macro SIUL_GPDI

Prototype: `SIUL_GPDI (PIN)`

10.1.2.2 Macro SIUL_GPDO

Prototype: `SIUL_GPDO (PIN)`

10.1.2.3 Macro SIUL_IFMC

Prototype: `SIUL_IFMC (IFMC_OFFSET)`

10.1.2.4 Macro SIUL_MPGPDO

Prototype: SIUL_MPGPDO (MPGPDO_OFFSET)

10.1.2.5 Macro SIUL_PCR

Prototype: SIUL_PCR (PIN)

10.1.2.6 Macro SIUL_PGPDI

Prototype: SIUL_PGPDI (PGPDI_OFFSET)

10.1.2.7 Macro SIUL_PGPDO

Prototype: SIUL_PGPDO (PGPDO_OFFSET)

10.1.2.8 Macro SIUL_PSMI

Prototype: SIUL_PSMI (PSMI_OFFSET)

10.1.3 Enum Reference

Enumeration of all constants supported by the Port driver.

10.1.4 Function Reference

Functions supported by the Port driver.

10.1.5 Structs Reference

Data structures supported by the Port driver.

10.1.6 Types Reference

Types structures supported by the Port driver.

10.1.7 Variables Reference

Variables supported by the Port driver.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13PORTASR4.0 Rev0003R1.0.1
Revision 1.2